

Lista de Código: Deploy e Variáveis de Ambiente

Turma: LionsDev

Tópicos: variáveis de ambiente (`process.env`), porta dinâmica (`process.env.PORT`), `.env` e `.gitignore` , script `start` no `package.json` e preparação da API para produção (Render).

Aqui você deixa a API pronta pra rodar fora da sua máquina. O trabalho é trocar valor chumbado no código por configuração vinda do ambiente, pra mesma API rodar local e em produção sem editar nada.

Parte 0 — Treino rápido (aquecimento)

Cada item tira um valor fixo do código e joga pro ambiente.

1. Imprima o valor de `process.env.PORT` .
2. Defina a porta com fallback: `const PORT = process.env.PORT || 3000` .
3. Leia a string de conexão: `const uri = process.env.MONGO_URI` .
4. Faça o app escutar em `PORT` (não em um número fixo).
5. Crie um arquivo `.env` com `PORT=3000` e `MONGO_URI=...` .
6. Adicione `.env` ao `.gitignore` (por que ele não pode ir pro Git?).
7. No `package.json` , crie o script `"start": "node index.js"` .
8. Leia o segredo do token: `process.env.JWT_SECRET` .
9. Crie uma rota `GET /health` que responde `{ status: "ok" }` .
10. Imprima no boot: `console.log("Servidor rodando na porta", PORT)` .

Parte 1 — Complete o código

1. Porta dinâmica

Complete para a API funcionar tanto local quanto no servidor.

```
// TODO: use a porta do ambiente, com 3000 como padrão
const PORT = /* ... */;

app.listen(PORT, () => {
```

```
console.log(`Servidor no ar na porta ${PORT}`);  
});
```

2. Conexão via ambiente

Complete a conexão lendo a URI do ambiente.

```
// TODO: pegue a string de conexão da variável de ambiente  
const MONGO_URI = /* ... */;  
  
await mongoose.connect(MONGO_URI);
```

3. Script de start

Complete o `package.json` para o Render saber como iniciar a API.

```
{  
  "scripts": {  
    "start": "/* TODO */"  
  }  
}
```

Parte 2 — Ache o problema

4. Porta chumbada

Esta API funciona na máquina do aluno mas quebra no Render. Por quê? Conserte.

```
app.listen(3000, () => console.log("rodando")); // problema em produção
```

5. Segredo no código

Este código vaza credenciais e vai parar no GitHub. Aponte o risco e diga como corrigir.

```
const MONGO_URI = "mongodb+srv://admin:senha123@cluster0.mongodb.net/loja";  
// BUG grave  
const JWT_SECRET = "meusegredo"; // BUG grave
```

Parte 3 — Prever o comportamento

6. Qual valor sai?

Suponha que o `.env` tem `PORT=8080` e não tem `MONGO_URI`. Diga o valor final de cada constante.

```
const PORT = process.env.PORT || 3000; // (a)
const URI = process.env.MONGO_URI || "local"; // (b)
const DEBUG = process.env.DEBUG; // (c)
```

Parte 4 — Prepare para o deploy

7. Checklist de Produção (Desafio)

Pegue uma API sua dos módulos anteriores e deixe-a pronta para deploy no Render. Entregue o código/config para cada item:

1. `PORT` lida de `process.env.PORT` com fallback.
2. `MONGO_URI` e `JWT_SECRET` lidas de `process.env` (nada chumbado).
3. Arquivo `.env` criado e listado no `.gitignore`.
4. Script `"start"` no `package.json`.
5. Rota `GET /health` respondendo `{ status: "ok" }`.
6. Um `README` curto explicando quais variáveis de ambiente configurar no painel do Render.

Ao final, descreva em 3 passos como você faria o deploy conectando o repositório do GitHub ao Render.

Dica: código vai pro Git, segredo vai pro ambiente. Nunca deixe porta, URI do banco ou segredo do token escritos no código; leia tudo de `process.env` e configure os valores no painel do serviço. O `.env` fica de fora do Git, sempre no `.gitignore`.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: Deploy e Variáveis de Ambiente - Módulo 10